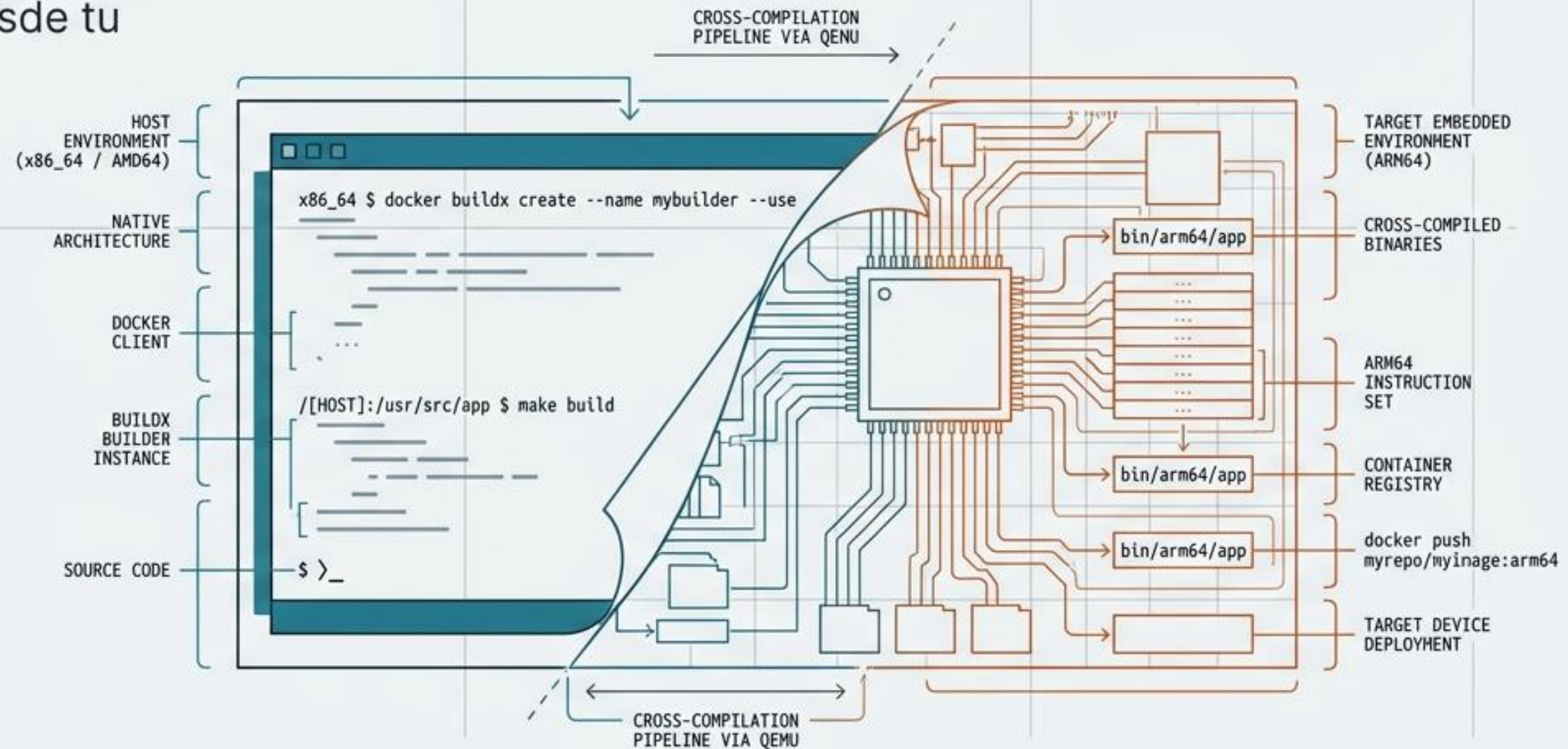


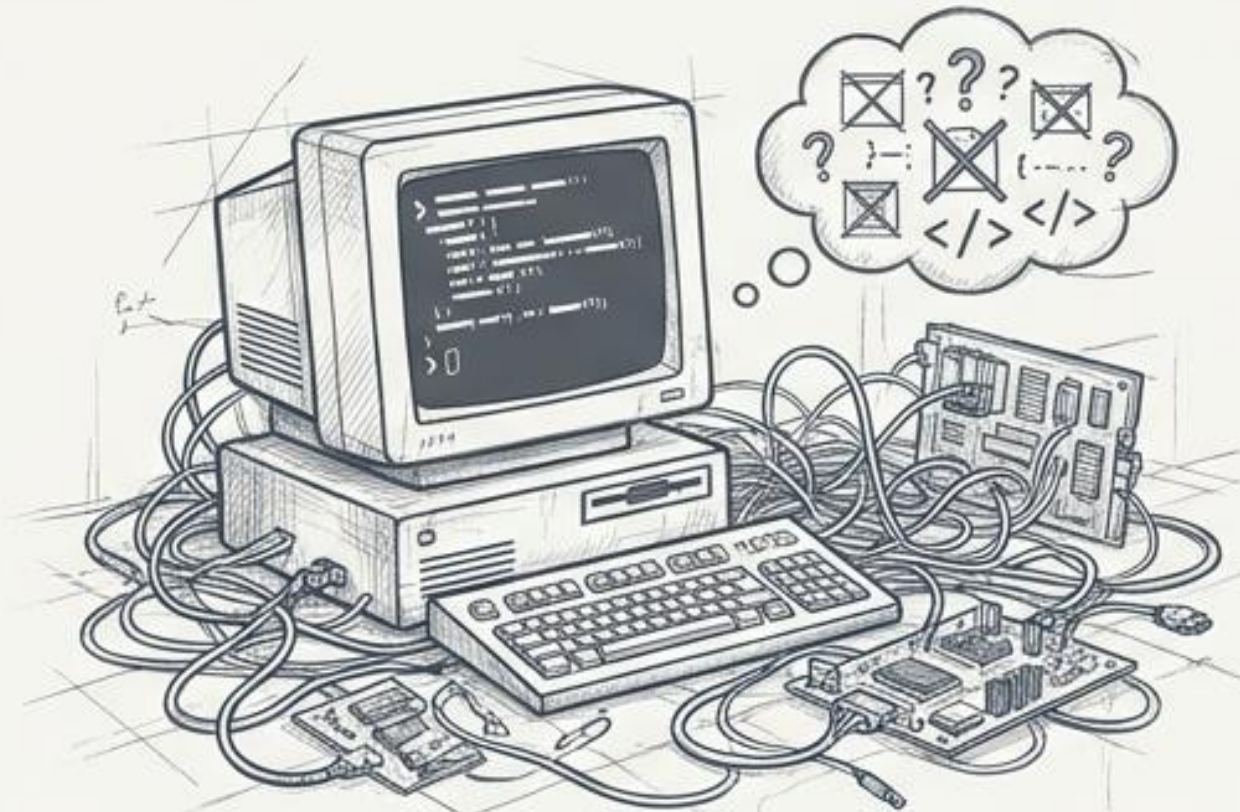
Desarrollo Embebido Multiplataforma

Cómo usar Docker y Buildx para crear software ARM64 nativo desde tu entorno x86_64.



El Paradigma de la Compilación Cruzada

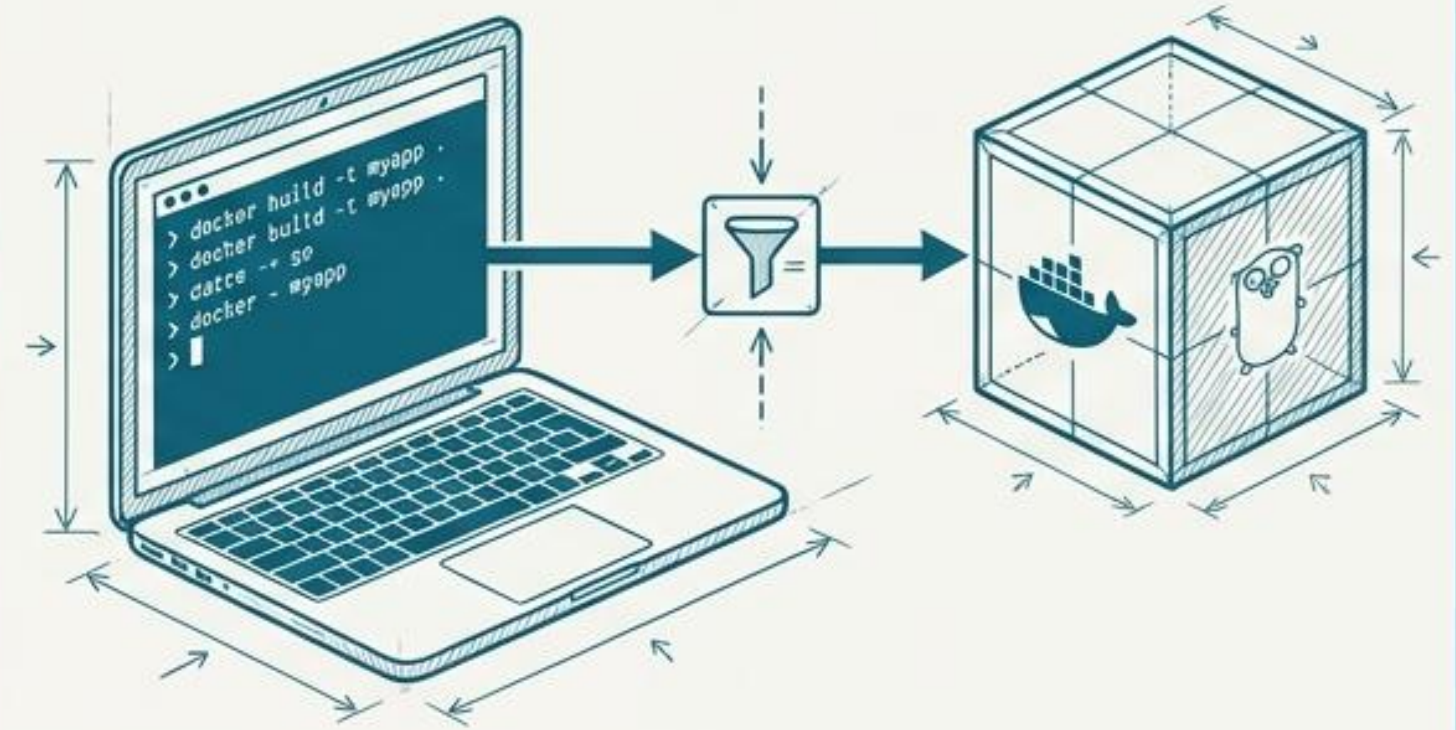
Path A - El Pasado



El Caos

Toolchains complejas, variables de entorno rotas y dependencias cruzadas instaladas directamente en el sistema local.

Path B - El Presente

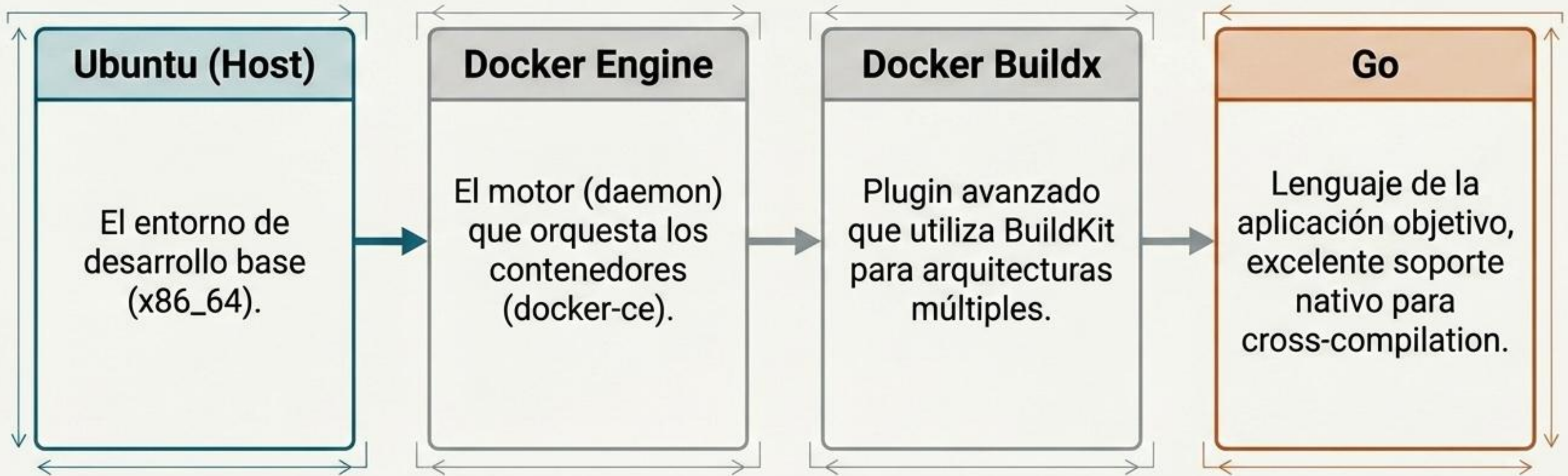


El Aislamiento

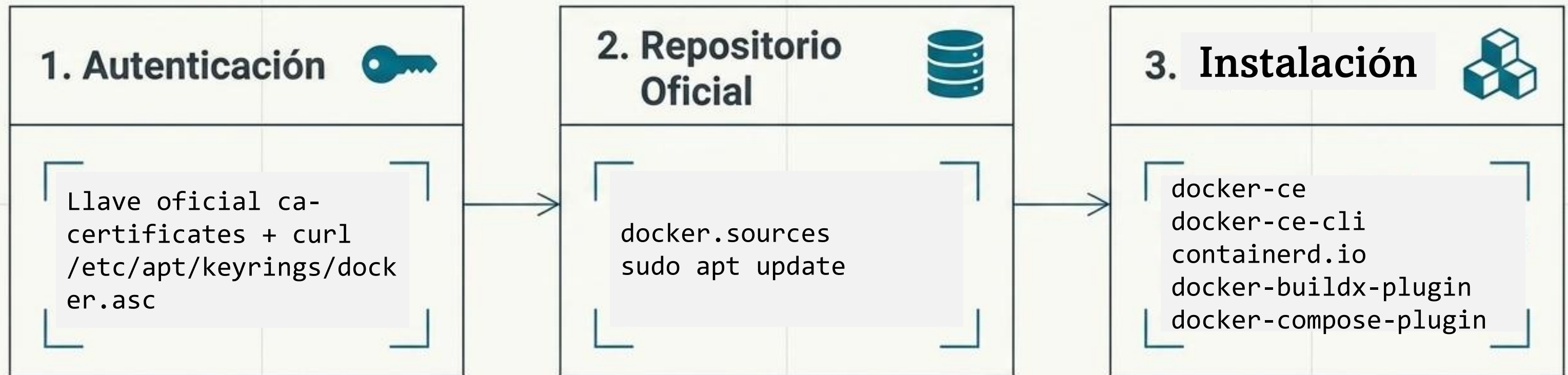
Docker + Go. Entornos de compilación controlados y builds multiplataforma.

El objetivo es comprobar que una **computadora de desarrollo x86_64/amd64** puede generar software destinado a ARM64 sin ensuciar el sistema base.

El Stack de Construcción

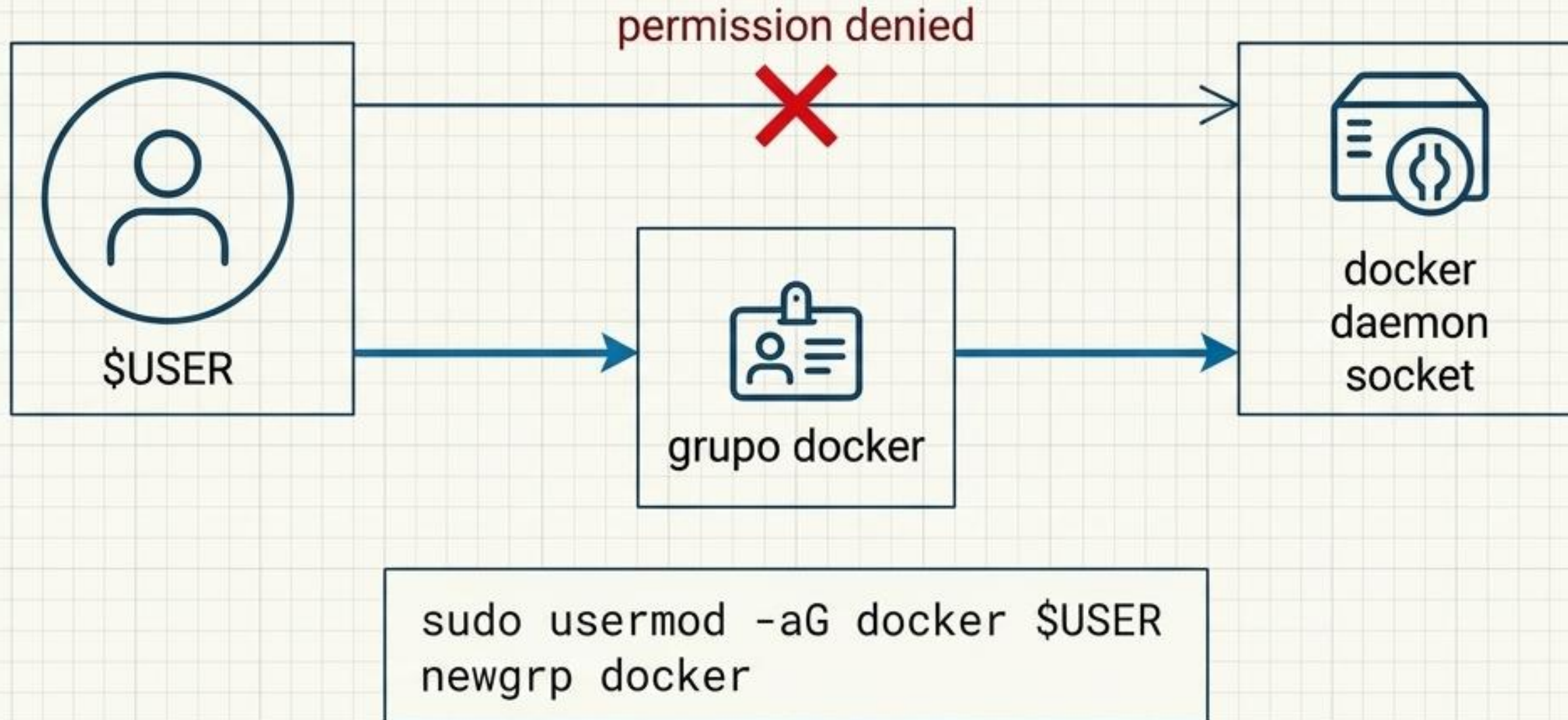


Pipeline de Preparación del Host



Instala Docker Engine, CLI, Runtime y el plugin vital: Buildx.

Fricción Cero: Docker sin sudo



Añadir el usuario al grupo docker otorga permisos directos al socket, agilizando el flujo de trabajo del desarrollador sin depender de superusuario.

La Aplicación Objetivo (main.go)

```
package main

import ( "fmt"; "runtime" )

func main() {
    fmt.Printf("OS: %s\n", runtime.GOOS)
    fmt.Printf("Architecture: %s\n", runtime.GOARCH)
}
```

El programa muestra el sistema operativo y la arquitectura para los que fue compilado. Esto permite comprobar el efecto de la cross-compilation.

La Magia Multiplataforma: El Dockerfile

```
FROM --platform=$BUILDPLATFORM golang:1.24-alpine AS build
```

```
ARG TARGETOS  
ARG TARGETARCH
```

```
RUN GOOS=$TARGETOS GOARCH=$TARGETARCH go build -o /out/app .
```

Ejecuta la etapa de compilación sobre la plataforma del builder.

BuildKit inyecta dinámicamente lo que pidas en tu comando docker buildx.

El compilador Go en AMD64 genera un binario alienígena para la arquitectura destino (ARM64).

BuildKit: El Motor Oculto



```
docker buildx create --name embedded-builder --driver docker-container --use
```

Etiqueta identificativa del constructor.

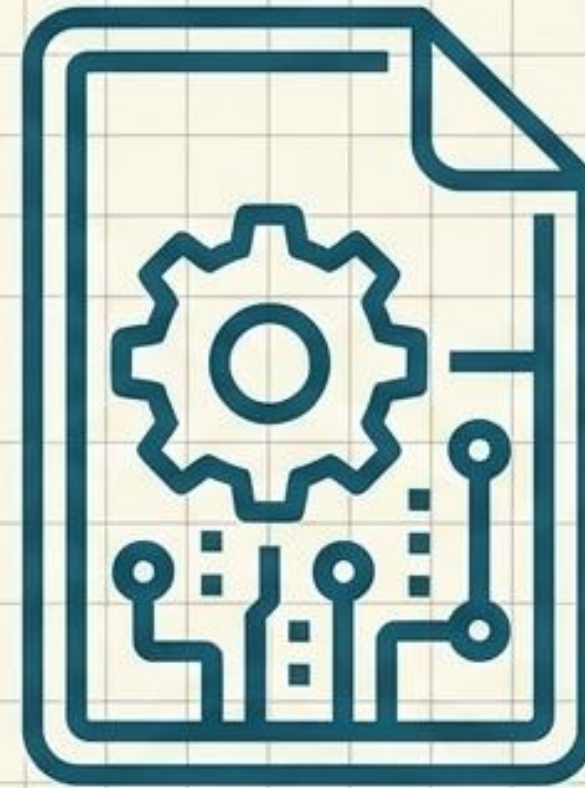
Fuerza a BuildKit a ejecutarse dentro de un contenedor aislado, permitiendo soporte multi-arquitectura avanzado.

```
embedded-builder * docker-container
```


Compilación 1: Binario Nativo (AMD64)

```
docker buildx build  
  --platform linux/amd64  
  --target artifact  
  --output type=local,dest=out-amd64 .
```

```
[+] Building ...  
=> exporting to client directory
```

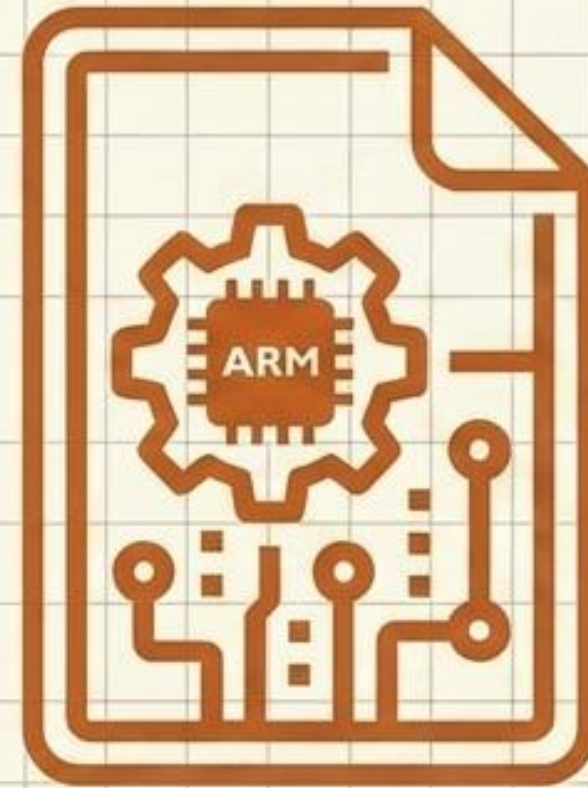


El Host y el Target coinciden. Compilación nativa rápida sin emulación cruzada.

Compilación 2: Binario Embebido (ARM64)

```
docker buildx build  
  --platform linux/arm64  
  --target artifact  
  --output type=local,dest=out-arm64 .
```

```
[+] Building ...  
=> exporting to client directory
```



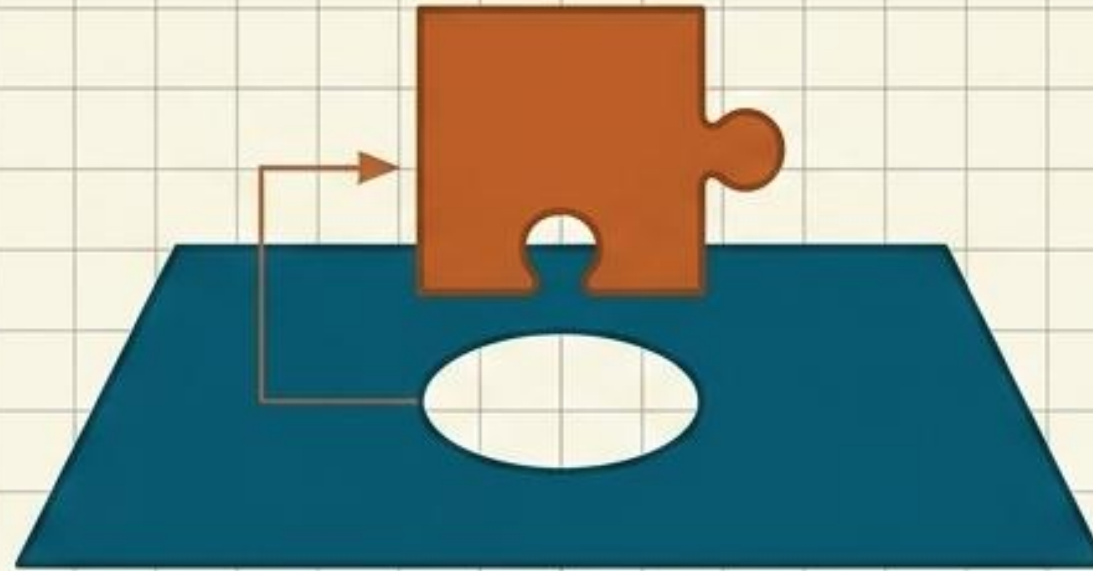
Mismo código fuente.
Mismo Dockerfile.
Diferente arquitectura
objetivo generada con
un solo parámetro.

Análisis Forense: Prueba Irrefutable

	out-amd64/app	out-arm64/app
Comando file	ELF 64-bit LSB executable, x86-64	ELF 64-bit LSB executable, ARM aarch64
Cabecera ELF (readelf -h)	Machine: Advanced Micro Devices X86-64	Machine: AArch64

Inspeccionar los binarios confirma físicamente el éxito de la compilación cruzada a nivel de sistema.

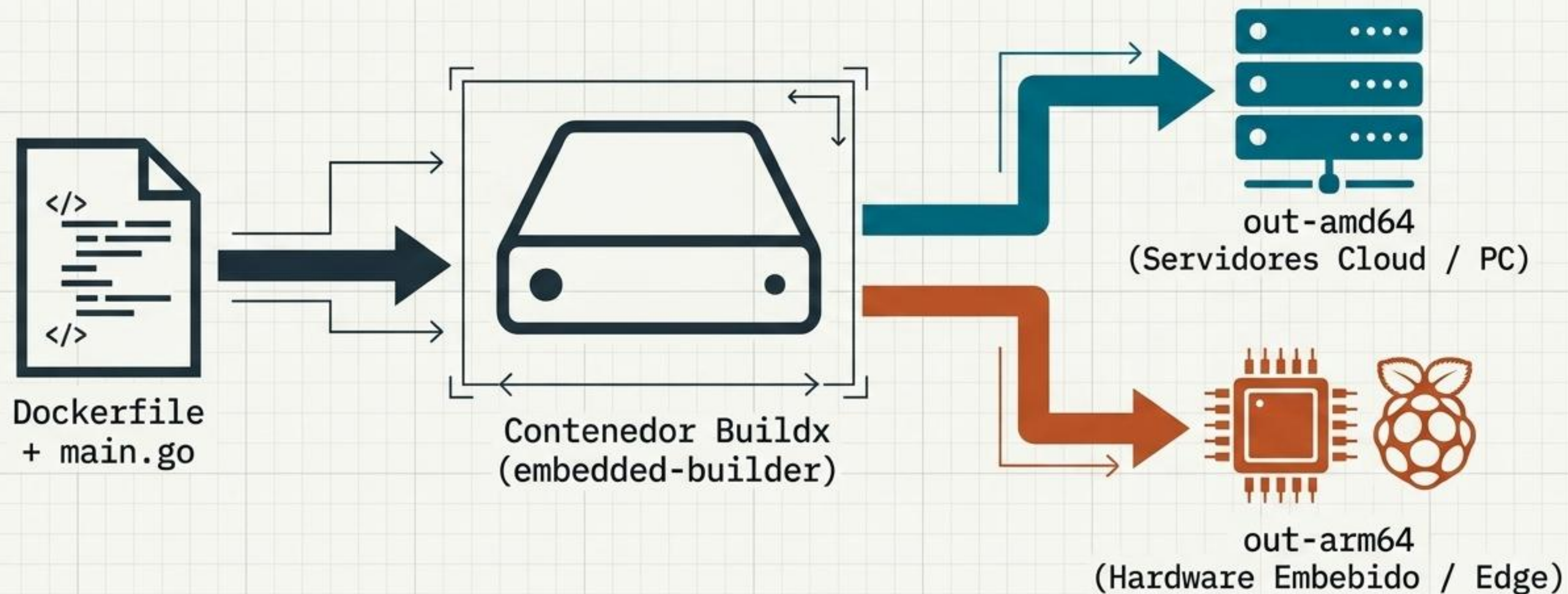
La Paradoja de la Ejecución



```
$ ./out-arm64/app  
bash: ./out-arm64/app: cannot execute binary file: Exec format error
```

¡El error es esperado y es un éxito! Ambos son binarios Linux (ELF), pero el Host (x86_64) carece de las instrucciones nativas (ISA) para procesar el código AArch64 que acabamos de compilar. La arquitectura es correcta.

Síntesis de Flujo: Code Anywhere, Build Everywhere



Un único código base y un entorno de construcción controlado pueden generar artefactos para distintas arquitecturas soportadas por el toolchain.

Validación del Entorno

- ✓ Motor Docker y Buildx instalados y operando en modo sin sudo.
- ✓ Entorno de construcción aislado (embedded-builder) activo.
- ✓ Aplicación construida para múltiples arquitecturas mediante Dockerfile.
- ✓ Binarios **AMD64** nativos compilados y verificados.
- ✓ Binarios **ARM64** embebidos compilados cruzados con éxito.

```
/tutorial-docker-cross-compile
├── Dockerfile
├── main.go
├── out-amd64/
│   └── app (x86_64)
└── out-arm64/
    └── app (AArch64)
```